FutureDID: A Fully Decentralized Identity System With Multi-Party Verification

Haotian Deng, *Student Member, IEEE*, Jinwen Liang, *Member, IEEE*, Chuan Zhang, *Member, IEEE*, Ximeng Liu, *Senior Member, IEEE*, Liehuang Zhu, *Senior Member, IEEE*, and Song Guo, *Fellow, IEEE*

***Abstract*—Decentralized identity (DID) systems conforming to the World Wide Web Consortium (W3C) Decentralized Identifiers (DIDs) and Verifiable Credentials Data Model recommendations have recently attracted attention due to their better autonomy, interoperability, and openness design. However, those W3C recommendations lack a design for addressing the single point of failure (SPOF) and identity revocation, which could seriously compromise the robustness and practicality of DID systems. To remedy these limitations, we propose FutureDID, a DID system that enables multiple parties to jointly issue credentials and efficiently revoke DID identities, providing a robust and practical DID system. FutureDID is designed with a multi-party credential issuing mechanism based on distributed key generation technology, which transforms trust from a single entity to distributed committees and facilitates authentication between issuers, making it more resistant to SPOF. Moreover, the underlying blockchain system is built on a chameleon hash function to ensure tamper-proof and enable efficient identity revocation. We have implemented a prototype system using FISCO BCOS and conducted extensive evaluations to demonstrate the effectiveness and practicality of our system. Our evaluations have shown that FutureDID provides a significant improvement in efficiency, achieving at least a 60 × efficiency improvement in identity revocation compared to state-of-the-art systems.**

***Index Terms*—Decentralized identity, blockchain, distributed key generation, chameleon hash function, identity revocation.**

Manuscript received 2 July 2023; revised 8 April 2024; accepted 19 April 2024. Date of publication 9 May 2024; date of current version 11 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62232002 and Grant 62202051, in part by the National Key R&D Program of China under Grant 2021YFB2700500 and Grant 2021YFB2700503, in part by China Postdoctoral Science Foundation under Grant 2021M700435 and Grant 2021TQ0042, in part by Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies under Grant 2022B1212010005, in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2021B0101400003, in part by the Open Project Funding of Key Laboratory of Mobile Application Innovation and Governance Technology, Ministry of Industry and Information Technology under Grant 2023IFS080601-K, in part by Yunnan Provincial Major Science and Technology Special Plan Projects under Grant 202302AD080003, in part by Beijing Institute of Technology Research Fund Program for Young Scholars, the Young Elite Scientists Sponsorship Program by CAST under Grant 2023QNRC001, in part by Hong Kong RGC Research Impact Fund under Grant R5060-19 and Grant R5034-18, in part by the Areas of Excellence Scheme under Grant AoE/E-601/22-R, and in part by the General Research Fund under Grant 152203/20E, Grant 152244/21E, Grant 152169/22E, and Grant 152228/23E. Recommended for acceptance by K. Pattabiraman. *(Corresponding authors: Jinwen Liang; Chuan Zhang.)*

Haotian Deng, Chuan Zhang, and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: hdeng@bit.edu.cn; chuanz@bit.edu.cn; liehuangz@bit.edu.cn).

Jinwen Liang is with the Department of Computing, Hong Kong Polytechnic University, Hung Hom, Hong Kong (e-mail: jinwen.liang@polyu.edu.hk).

Ximeng Liu is with the College of Mathematics and Computer Science, Fuzhou University, Fuzhou 350108, China (e-mail: snbnix@gmail.com).

Song Guo is with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Kowloon, Hong Kong (e-mail: songguo@cse.ust.hk).

Digital Object Identifier 10.1109/TC.2024.3398509

I. Introduction

DIGITAL identity management [1] refers to the processes and technologies used to manage and protect the digital identities of individuals, organizations, and other entities online. It is becoming increasingly important in today's digital world and plays a crucial role in ensuring the security and privacy of digital transactions and interactions in e-commerce, financial services, healthcare, government, etc. However, traditional digital identity systems are plagued by numerous accounts with non-standardized identities, centralized architecture, and applications that store personal identity data which are often unreliable and incompatible. For example, OpenID [2], a widely-used digital identity management system, could allow attackers to impersonate users and gain access to their accounts. In 2019, Estonia experienced a security incident [3] in its digital identity system, which affected 750,000 residents.

To address these challenges, the decentralized identity (DID) technology [4] has emerged as a potential solution. In DID systems, individuals become the stewards of their digital identity, allowing them to control their identity data. DID technology guarantees the authenticity and reliability of data, safeguards user privacy and security, and offers strong portability. The key advantages of DID technology include:

- Decentralization: DID technology leverages blockchain technology to ensure that identity data is not controlled by a single centralized authority. This reduces the risk of data breaches and ensures that individuals have greater control over their personal information.
- Self-sovereign identity: DID technology allows each user to control their own identity rather than relying on a trusted third party. This approach enables individuals to manage their own identity independently.
- Trusted data exchange: Identity-related data is anchored on the blockchain, and the authentication process does not depend on the application party providing the identity. Effectively prevent authentication information from being maliciously tampered with.

0018-9340 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

Given these properties, DID technology presents a promising solution for identity management in Web3 and the meta-verse, which are based on decentralized network architectures. Both industry and academia attach great importance to DID systems. Thus, the Decentralized Identifiers working group of the World Wide Web Consortium (W3C) published recommendations [4], [5] and use cases to standardize DID systems. However, we have identified limitations in W3C recommendations related to credential issuance and identity revocation. Specifically, the current W3C recommendations allow a credential to be issued by a single issuer, and the verifier only verifies the issuer's signature without further authentication. It presents the Single Point of Failure (SPOF) risks to DID systems. Because DID systems' security and robustness could be compromised when the issuer is compromised or the malicious issuer is involved in the issuance process. Besides, although the revocation mechanism is mentioned in W3C recommendations [4], [5], there is no available method for identity revocation. Nevertheless, the-state-of-art schemes [6], [7], [8], [9], do not perfectly address these limitations, the specific challenges are as follows:

1) *Single point of failure*: The SPOF is highly likely to break down DID systems. Several works attempt to solve this problem. WeIdentity [7] proposes a mechanism that builds an authority issuer list to verify whether the issuer is reliable, but the mechanism requires a trusted third party to manage the list, which contradicts the decentralized nature of DID systems and it also results in high communication and storage overhead. CanDID [9] proposes the establishment of a committee as the issuer of credentials. However, CanDID [9] only has one committee, and all credentials are issued by this one committee, which leads CanDID to be a semi-centralized structure. Additionally, since CanDID [9] employs a key recovery function, the committee is responsible for backing up the user's most sensitive private key. This means that the committee cannot be replaced once it is established. Thus, CanDID [9] still introduces a risk of SPOF.
2) *Identity revocation*: Identity revocation has a strong practical need. On the one hand, DID systems need to delete the duplicate identity registered by the user on the platform [1], on the other hand, there are certain situations [10] where identities need to be revoked by DID systems. These include instances where the user is sanctioned by the government [11], has their degree revoked [12], or due to other special circumstances [13]. Identity revocation is a difficult challenge to resolve in DID systems [14] because there is no central server that can simply revoke users' cryptographic keys [15]. Additionally, blockchain [16], [17] is the underlying technology for DID systems, and is immutable in its functionalities, making revocation even more challenging. Besides, many state-of-the-art DID systems [7], [8], [9] do not perfectly address the problem of identity revocation. For example, CanDID's [9] revocation mechanism involves maintaining a public sanctions list, which requires querying the list and providing non-existent proof. it relies on centralized storage, resulting in significant storage and communication overheads.

FutureDID offers a pragmatic solution to the challenges of decentralized identity, effectively addressing and overcoming numerous substantial challenges. Our main technical contributions are:

- **Single point of failure resistance:** FutureDID utilizes distributed key generation technology to implement a multi-party joint credential issuance method, which enhances decentralization and reduces the risk of SPOF. It also incorporates a multi-credential committee mechanism, selecting a committee related to the type of the users' claim to ensure fairness and impartiality during the credential issuance process.
- **Mutual verification**: FutureDID's multi-credential committees carry out byzantine consensus when issuing credentials. The consensus result is based on the verification by the committee members on the ability and identity of the issuer. The credential can only be issued when the number of committee members who have passed the verification is greater than or equal to 2/3 of the committee's qualifications.
- **Practical revocation mechanism**: FutureDID is able to revoke the identity of malicious or illegal users with no additional storage and much less time overhead.
- **Implementation and evaluation**: We conducted extensive experiments and analysis on FutureDID to evaluate its performance and effectiveness. Additionally, we compared our system comprehensively with state-of-the-art works to demonstrate its superiority. FutureDID achieves at least a 60 $\times$ efficiency improvement in identity revocation compared to CanDID [9].

II. Related Work

Recently, some decentralized identity management systems have been proposed [7], [8], [9], [18], [19], [20], [21]. Specifically, uPort [18] is a decentralized identity system built on the Ethereum blockchain. As it is primarily based on Ethereum, interoperability issues may arise. In contrast, WeIdentity [7] is a decentralized multi-centered technology solution that provides a series of standardized interfaces to support cross-chain and cross-platform interoperability, but WeIdentity has limitations when it comes to security and privacy issues. Therefore, some schemes focus on security during the authentication process. RSS-TMRC [20] is a multi-party control signature scheme without leaking any sensitive information, but it relies on the complicated tree structure. Although Liu et al. [19] proposed a blockchain-based decentralized authenticated information-sharing scheme that supports the zero-trust environment, its communication latency is relatively high. Furthermore, some scholars have put forward designs tailored to specific fields. For instance, SmartDID [8] is a decentralized identity management scheme created specifically for Internet of Things (IoT) decentralized systems. Nevertheless, these solutions are not generalizable since they are designed for specific fields. A key challenge of distributed identity systems is revocation, as most

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FUNCTIONALITY COMPARISON WITH RELATED WORKS

Scheme	Decentralization	Scalability	Identity revocation	Mutual verification	Single point of failure resistance
Uport [18]	●	●	○	○	○
WeIdentity [7]	●	●	○	○	◐
SmartDID [8]	●	●	○	●	◐
CanDID [9]	●	●	◐	●	◐
FutureDID	●	●	●	●	●

●: Function is available, ◐: Limited availability of this function, ○: Function is not available.

schemes are based on distributed ledgers like blockchain technology, which is difficult to modify. Several identity systems [9], [22], [23] have proposed revocation solutions, but these approaches pose risks for non-technical users. For example, in the scheme proposed by Guo et al. [23], the user needs to use their partial key to retrieve the user's master key in conjunction with the blockchain and is required to collaborate with the cloud server for revocation purposes. In addition, CanDID [9] proposes a revocation mechanism, but it is expensive in computation and storage.

Functionality comparison. We compare the functionality of FutureDID with WeIdentity [7], uPort [18], and CanDID [9]. These systems were chosen because they are representative. uPort [18] and WeIdentity [7] are both open-source commercial projects, where uPort is based on Ethereum and WeIdentity [7] is based on the consortium blockchain. SmartDID [8] and CanDID [9] are state-of-the-art academic research. As shown in Table I, uPort [18] does not have the functions of identity revocation, mutual verification, and SPOF resistance. WeIdentity [7] lacks the ability to revoke identity, it also has limitations in resisting single points of failure, since members of the authority list can perform malicious attacks. SmartDID [8] does a lot of work on privacy protection but is not designed for identity revocation. CanDID [9] only has one committee, and all credentials are issued by this committee, which becomes a semi-centralized structure. It should be noted that since CanDID [9] designed the key recovery function, it cannot be replaced once the committee is established. As a result, there is still a risk of SPOF within CanDID. If the committee is compromised, the entire system could collapse. Additionally, the fact that all credential issuance depends on the same committee increases the risk of SPOF. Besides, It is important to emphasize that CanDID supports key recovery and Sybil-resistant functionalities primarily based on committee mechanisms and secure multi-party computation (MPC) technologies [24]. FutureDID, with its multi-credential committee and administrative committee mechanisms, is compatible with CanDID's committee mechanism and can also execute MPC algorithms. Therefore, FutureDID can perfectly integrate and support CanDID's functionalities without the need for any additional mechanisms or entities. Moreover, CanDID [9] has limitations when it comes to revoking identities. It relies on sanction lists, and the larger the sanction lists, the larger the query overhead. If the data size exceeds 10,000 strings, the overhead becomes unacceptable. In summary, to the best of our knowledge, except for FutureDID, none of the DID systems can fully meet all the functionalities in Table I.

III. PRELIMINARIES

A. Decentralized Identity System

Decentralized identity (DID) [4], [5] is a new authentication system based on blockchain technology [16], [17]. The core idea of DID systems is to solve the problems of traditional authentication systems by decentralizing the storage and management of personal identity based on distributed ledgers. DID systems can transform personal identity credentials into digital identity credentials that only the identity user controls. At the same time, DID systems can realize the exchange and verification of identity information, ensure the authenticity and integrity of identity information, and protect personal privacy.

1) DID Terminology: **DID and DID URL**. A DID is a Uniform Resource Identifier (URI) that has the following format: did:method: identifier (e.g. *did:btcr: xyv2-xzpq-q9wa-p7t*), where a method is usually synonymous with the blockchain that the DID belongs to.

DID document. The DID document is a JSON-formatted file that contains information about the DID subject. In addition, the DID document can also contain service endpoints, which are typically URLs pointing to external sources related to the DID subject. This enables the DID subject to provide access to services or resources associated with their digital identity.

2) Verifiable Credential: Verifiable credential (VC) [5] technology is an important part of the DID technology operating on four entities, involving Issuer, Holder, Verifier, and DID registration service. As shown in Fig. 1, the holder is an entity that possesses one or more VCs and generates verifiable presentations from them. The issuer is an entity that asserts claims about one or more subjects, creates VCs based on these claims and transmits the verifiable credentials to the holder. A verifier is an entity that receives one or more VCs (possibly within a verifiable presentation) for processing. The role of a registry is to create and validate DIDs, keys, and other related data in order to use verifiable credentials. The registry is typically implemented using blockchain technology. For the sake of description, the term "credential" mentioned in the following text refers to the verifiable credential.

B. Distributed Key Generation

Distributed key generation (DKG) techniques [25] is the main component of the threshold cryptosystem, which mainly calculates the shared public and private key sets through the form of multi-party participation without relying on any trusted third party, and its core idea is (t, n) threshold secret sharing.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

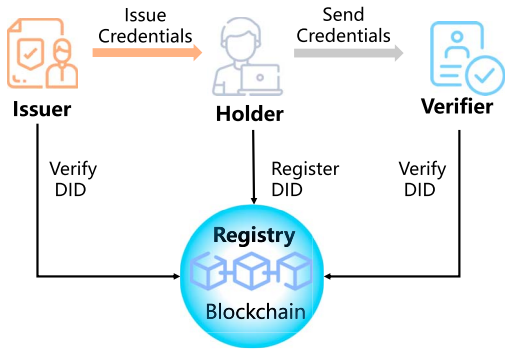


Fig. 1. Verifiable credential mechanism.

Traditional key generation methods involve a single entity, such as a central authority or trusted party, generating and distributing the keys. However, these centralized methods are prone to security vulnerabilities and can lead to a complete breach of the system if compromised. DKG overcomes this challenge by distributing the key generation process among multiple parties responsible for generating and managing the keys.

C. Chameleon Hash Function

The concept of chameleon hashing was put forward by Krawczyk and Rabin [26], building on the notion of chameleon commitments. This hash function contains a trapdoor private key that allows users with access to this key to calculate effective collisions for any hash value. FutureDID is based on the chameleon hash function with ephemeral trapdoors proposed by Camenisch et al. [27]. The chameleon hash function with ephemeral trapdoors is comprised of five algorithms.

- $Setup(\lambda \rightarrow pp)$: Parameter generation algorithm. Take parameter λ as input and output public parameter pp.
- $KeyGen(pp \rightarrow \{pk, sk\})$: Key generation algorithm. With the public parameter pp as input, the output key pair (pk, sk) is the public key is pk and the private key is sk.
- $Hash(pk, m, r \rightarrow hash)$: Hash algorithm. Take public key pk, message m, and chameleon random number r as input, and output chameleon hash value $hash$.
- $Hashcheck(pk, m, r, hash \rightarrow \{0 \textbf{ or } 1\})$: Hash check algorithm. Take public key pk, message m, chameleon random number r, and chameleon hash value $hash$ as input. If the verification, the output is 1, otherwise, the output is 0.
- $Adapt(sk, m, r, m' \rightarrow r')$: Take private key sk, message m, chameleon random number r and new content m' as input, output new random number r'.

IV. System Overview

A. System Model

As illustrated in Fig. 2, FutureDID is a system that comprises three types of entities: User (U), Issuer (I), and Verifier (V). FutureDID contains two types of committees, namely the multi-credential committee (MC) and the administration committee (AC). Moreover, the lifecycle of a credential can be divided into two phases, which are implemented by identity and revocation subsystems.

1) **User (U):** U is a registered user in FutureDID, who possesses one or more VCs and generates presentations from them. U typically stores and manages VCs by himself.
2) **Issuer (I):** I is an entity empowered to check users' claims, issue VCs based on these claims, and transmit VCs to users.
3) **Verifier (V):** V can verify the DIDs and VCs provided by users, thereby ensuring the authenticity of their DIDs and the validity of the VCs.
4) **Multi-credential Committee (MC):** MC is composed of multiple credential committees (CCs). Different CCs are responsible for issuing different types of VCs. The MC plays a crucial role in verifying the issuance of VCs and ensuring the competence and identity of committee members. This design incorporates a mutual verification mechanism, whereby MC members mutually verify in the credential issuance process. MC acts as issuers in FutureDID.
5) **Administration Committee (AC):** AC is mainly composed of system management parties. The committee uses the byzantine consensus protocol [28] and chameleon hash function [26] to revoke DIDs.

B. Workflow of System

As shown in Fig. 2, FutureDID consists of two subsystems, which are the identity subsystem (blue brackets) and the revocation subsystem (orange brackets).

Identity subsystem. Firstly, the U registers his identity with the blockchain. After registration, U makes a claim about his one attribute, such as the undergraduate degree, and submits the claim to the I to apply for a VC. Then, I verifies whether U's DID is valid from the blockchain. If the DID is valid, the identity subsystem establishes a MC with the I. MC performs the mutual verification operation to ensure I can issue the undergraduate degree VC for U. After passing mutual verification, U gets his VC. When U presents his VC to V, V verifies MC's signature on the VC and U's DID. If the verification is successful, U is authenticated and can proceed with his intended action. Technical details of the identity subsystem design are in Section V-A.

Revocation subsystem. If a trusted authority in the real world submits a revocation request of U's DID to AC, the request is processed through a consensus mechanism. If the consensus passes, AC performs the revoke operation to modify or delete the relevant DID in the blockchain. When V attempts to verify U's DID, it will find that U's DID no longer exists in FutureDID, which will result in failed authentication. Technical details of the revocation subsystem design are in Section V-B.

C. Adversary Model

We specifically give the adversary's ability to attack FutureDID:

- The adversary $\mathcal{A}$ can eavesdrop, forge, modify, delay, or delete messages between any two communicating entities in the FutureDID.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

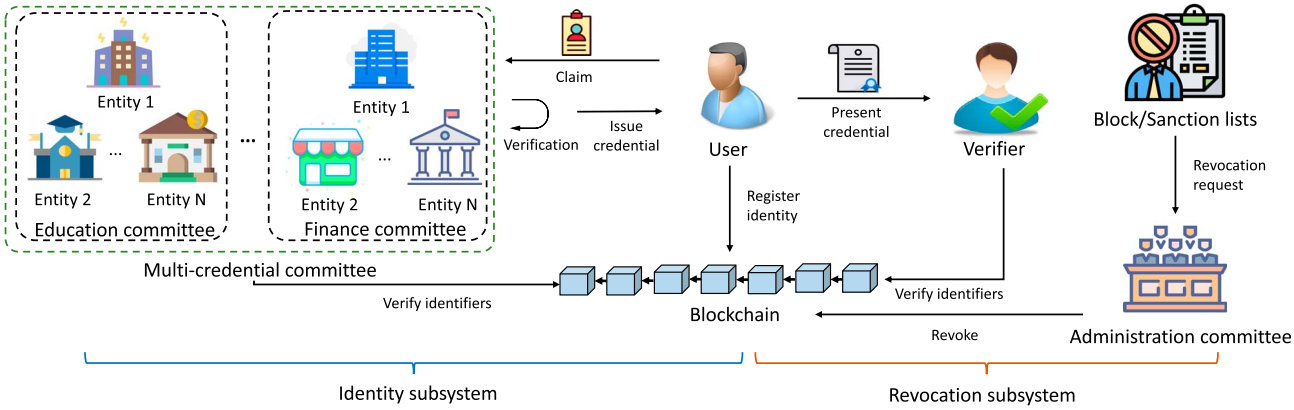


Fig. 2. System model of FutureDID.

- The communicating entities are not considered trusted, which means that these entities may be compromised by the adversary $\mathcal{A}$.
- The adversary $\mathcal{A}$ can access the blockchain network and get all the information stored on the blockchain, but $\mathcal{A}$ cannot modify the entity registration information stored in the blockchain ledger.
- We assume the adversary $\mathcal{A}$ to corrupt up to f of the n committee members, for $f < n/3$.
- The all entities of FutureDID are the honest majority.

V. System Design

FutureDID consists of two subsystems: the identity subsystem and the revocation subsystem.

A. Identity Subsystem

We now provide a detailed construction of the identity subsystem. The primary objective of this subsystem is to issue credentials to users with MC authentication.

Register DID. Users are required to present a valid identity attribute value v from the real world into the FutureDID for registering their DID, such as Social Security Numbers (SSN) value in the United States, for Sybil-resistance. In particular, the AC sets up an IDTable to store the user's real-world SSN attribute value v. User U_a generates his public key pk_{U_a} and private key sk_{U_a}. Then, U_a sends (pk_{U_a}, v) to FutureDID. Next, AC carries out the identity registration deduplication process using CanDID's deduplication policies [9] to check whether the U_a's v is already stored in the IDTable. If U_a is already registered, then AC prohibits him from repeating registration, if not, AC registers U_a's identity based on his public key pk_{U_a} and stores his v in IDTable. Finally, FutureDID generates U_a's DID DID_{U_a} (e.g. did: futureid: 123456abcdefg) and sends it to U_a.

Create Claim. U_a creates the claim $claim$ about his one attribute $claim$={att, val, DID_{U_a}}, which contains three parameters. The att is a string denoting what the claim is about, e.g., "Name.", val is the value of the attribute and DID_{U_a} is U_a's DID.

Select Committee. FutureDID selects a CC from MC to issue credentials when U_a applies the credentials to a certain issuer. The selection of the CC is based on the type of credentials applied by the users.

Mutual Verification. The credibility and reliability of credentials are critical aspects of digital identity systems. While some issuers are reputable institutions with a wealth of knowledge about their users, such as schools or hospitals, there still exists a risk of malicious issuers creating phishing websites and issuing fake credentials that can deceive verifiers under the current DID trust model [5]. In addition, some issuers may overstep their authorized privileges and issue credentials beyond their jurisdiction, such as a middle school issues a high school degree credential. These problems highlight the need for strong identity verification mechanisms to prevent fraudulent activities and protect the security of DID systems. Therefore, once a specific CC from MC is selected by FutureDID based on the type of credential to be issued, mutual verification is conducted among CC members to validate their identities, capabilities, and knowledge. If the members of CC successfully pass the mutual verification process, the DKG algorithm will be executed correctly during the credential issuance. However, if the verification fails for any CC member, the execution of the DKG algorithm is rejected. If the number of rejected members exceeds the threshold value t, the issuance of the credentials cannot proceed. This indicates that there are problems with the identities or insufficient capabilities of the CC members to issue specific credentials for users.

Credential Issuance. CC generates a credential $cred$ for the U_a. Then, CC sends the $cred$ to U_a, and the $cred$ is stored by the U_a himself. The details are as follows.

Step 1: Given a security parameter λ, FutureDID exposes parameters $params$. The parameters of FutureDID are

$$params = \{p, q, g, Z_q, n, t, H(\cdot)\}, \tag{1}$$

where p, q are large prime numbers, Z_q is the finite field, g is the q order generator of Z_q, n is the number of credential committee members, t is the threshold value for key generation, and the selected hash function is $H(\cdot)$.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

Step 2: Each credential committee member P_i selects t random numbers $a_{i0}, a_{i1}, \ldots, a_{i(t-1)}$ on Z_q, where $i = 1, ..., n$. P_i's identifier is ID_{P_i}.

Step 3: P_i constructs the function

$$f_i(x) = a_{i0} + a_{i1}x + ... + a_{i(t-1)}x^{t-1} (\text{mod } q). \quad (2)$$

Step 4: P_i computes $w_i = g^{a_{i(t-1)}} \bmod p$ and broadcasts. Then, all CC members compute $W = \prod_{i=1}^{n} w_i$. If $W = 1$, all CC members re-select the coefficients $a_{i(t-1)}$ in their $f_i(x)$, and repeat the above operation until $W \neq 1$.

Step 5: P_i according to Eq. 2 computes $f_i(ID_{P_i})$, $i = 1, ..., n$. P_i sends $f_i(ID_{P_j})$ to P_j $(j = 1, ..., n, j \neq i)$ via secure secret sharing, and P_i stores $f_i(ID_{P_i})$.

Step 6: P_i selects random number $k_i \in [1, p-1]$ to compute Eq. 3 and then broadcasts A_i,

$$A_i = g^{k_i} (\text{mod } p), k = 1, ..., t-1. \quad (3)$$

Step 7: We first define a function $F(x) = \sum_{i=1}^{n} f_i(x) \bmod q$, and then we can have $F(ID_{P_i}) = \sum_{j=1}^{n} f_j(ID_{P_i}) \bmod q$. P_i's private key is $sk_i = F(ID_{P_i})k_i \bmod q$, public key is $pk_i = g^{sk_i} \bmod p$.

Step 8: P_i computes $A = \prod_{i=1}^{n} A_i \bmod p$. Since the threshold is t, at least t members are required to recover CC's public key PK using the lagrangian interpolation method [29]. Each member computes PK,

$$L = \sum_{i=1}^{t} F(ID_{P_i}) \prod_{j=1, i\neq j}^{t} \frac{-ID_{P_j}}{ID_{P_i} - ID_{P_j}} \bmod q \quad (4)$$

$$PK = g^L A \bmod p.$$

CC's secret key is $SK = L + \sum_{i=1}^{n} k_i \bmod q$.

Step 9: P_i chooses random number $c_i \in [1, q-1]$, computes and broadcasts $r_i = g^{c_i} \bmod p$, $z_i = g^{c_i k_i^{-1}} \bmod p$, where k_i^{-1} is the inverse element of k_i on the field Z_q.

Step 10: P_i computes $r = \prod_{i=1}^{n} z_i \bmod p$, and solves s_i,

$$s_i = H(claim) F(ID_{P_i}) C_i - r c_i {k_i}^{-1} \bmod p, \quad (5)$$

where $C_i = \prod_{j=1, i\neq j}^{t} \frac{-ID_{P_j}}{ID_{P_i} - ID_{P_j}}$.

Step 11: After all CC members have computed their partial signatures (r, s_i), CC computes

$$s = \sum_{i=1}^{n} s_i \bmod q. \quad (6)$$

CC generates the signature $\sigma = (r, s, A)$, and the credential $cred = \{claim, \sigma, DID_{U_a}, PK\}$ for the U_a.

Verify Credential. U_a sends the $cred$ and public parameters $params$ to the verifier. Then, the verifier checks the validity of the $cred$. The verifier verifies whether the following equation,

$$g^s r^r \equiv (PK \times A^{-1})^{H(claim)} \bmod p, \quad (7)$$

holds, where A^{-1} is the inverse element of A on the field Z_q. If the equation holds, the authentication is passed otherwise, the authentication fails.

Key Recovery. In order to enhance the security and usability of the FutureDID, we have also implemented a key recovery mechanism based on CanDID [9]. This feature allows users to recover their cryptographic keys in the event of key loss or device failure. When the user U_a enrolls in key recovery service in FutureDID, U_a is required to provide his key K_a and a recovery policy to AC. AC securely stores K_a in an encrypted format by using an MPC [24] encryption algorithm. In the event of K_a loss or device failure, U_a can initiate the key recovery process. U_a will be required to provide certain identifying information (e.g., personal biometric information, Facebook account, SSN) to meet his recovery policy. Then U_a goes through the verification process to ensure he has ownership of a certain DID. Once the U_a's identity is verified, the recovery process is initiated. AC retrieves the encrypted K_a from the secure storage and decrypts the encrypted K_a by the MPC technology [24]. Then, the recovered K_a is securely transferred to the user, allowing U_a to regain access to FutureDID. It is worth noting that the key recovery process is designed with a strong emphasis on security. The encrypted K_a is stored separately from the user's identity information, adding an additional layer of protection.

B. Revocation Subsystem

1) Overview: In the real world, there are a large number of sanctions and prohibitions imposed on users by governments, industries, companies, etc. The state-of-the-art technologies, such as CanDID [9], still require the DID system to maintain a sanctions list and require tremendous computational overhead to prove that the user is not on the list, which however introduces serious storage overheads and computational overheads and, more seriously, run counter to decentralization. FutureDID utilizes the chameleon hash function [26] to replace the traditional hash function in the blockchain to make the blockchain editable. Therefore, FutureDID can eliminate the DID of illegal or sanctioned users directly from the blockchain without storing and querying sanction lists. Specifically, to remove the identity information of specific users from the blockchain, AC creates a new block that excludes their identity information. This new block serves as a replacement for the original block that contains the illegal user's identity information. To revoke identities, the following four steps are required to be followed: **System Setup**, **Block Generation**, **DID Rewriting**, **DID Revocation**. By following these steps, AC can ensure that the illegal user's identity information is removed from the blockchain and that his access permission to FutureDID is revoked.

2) Detailed Design: In this section, we provide a detailed description of the identity revocation subsystem, with the key algorithmic processes depicted in Fig. 3 below.

System Setup. To achieve the editability of the blockchain, we use the chameleon hash function rather than the normal hash function to establish the identity revocation subsystem, which can be seen in Algorithm 1. Specifically, the identity revocation subsystem first generates a series of public parameters, including the trapdoor private key sk and the hash public key pk of the chameleon hash function. The pk is then distributed to each node. Note that pk is obtained whenever a new user joins the blockchain, sk is only owned by AC in FutureDID.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

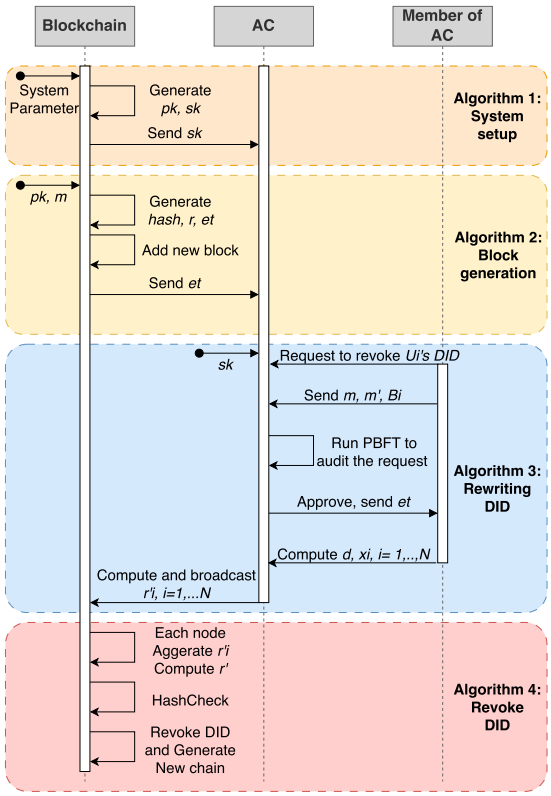


Fig. 3. Identity revocation process for each algorithm.

Algorithm 1: System setup

Input: System parameter λ, number of committee nodes N, field $\mathbb{Z}_p$.

Output: Public key pk, private key sk, public parameter $param$.

Select prime e.
Generate $param = (e, N)$.
Select prime $q \in \mathbb{Z}_p$ and $p \in \mathbb{Z}_p$.
Compute $n = pq$ where $gcd(n, e) = 1$.
Generate $sk \leftarrow (p, q)$, $pk \leftarrow n$.
return $param$, sk, pk.

Block Generation. After the underlying blockchain is built, users register their identities on FutureDID, and then the blockchain will generate a new block. The process of generating a block is described in Algorithm 2. Specifically, when a new block is created in the blockchain, the ephemeral trapdoor et needs to be sent to AC for secret storage, the chameleon hash value $hash$ and the chameleon hash random number r corresponding to the new block need to be broadcast.

DID Rewriting. When a member of AC finds that the DID_{U_i} is in the sanction or prohibition list, and the U_i's data is located in block $\mathcal{B}_j$, this member can initiate an edit request, and become an edit initiator. Then, AC runs Byzantine consensus

Algorithm 2: Block generation

Input: Identity message m, field $\mathbb{Z}_p$, chameleon hash function $H_n[\{0,1\}^* \to \mathbb{Z}_p]$, public hash key pk.

Output: Chameleon hash $hash$, chameleon random number r, ephemeral trapdoors et.

Select prime $q' \in \mathbb{Z}_p$ and $p' \in \mathbb{Z}_p$.
Generate $et \leftarrow (p', q')$, where $gcd(n, n') = 1$, $gcd(e, n') = 1$.
Select random number $r \in \mathbb{Z}_p$.
Generate $g \leftarrow H_n(m)$, $h \leftarrow gr^e (\mathrm{mod}\ nn')$,
$hash = (h, n')$.
return et, r, $hash$.

Algorithm 3: Rewriting DID with chameleon hash

Input: Number of block $\mathcal{B}_j$, original identity message m, new identity message m', chameleon hash value $hash$, chameleon hash ephemeral trapdoor key et, private key sk, number of committee nodes N, $H_n[\{0,1\}^* \to \mathbb{Z}_p]$.

Output: New chameleon hash random number r'_i.

Compute $\varphi(nn') = (p-1)(q-1)(p'-1)(q'-1)$.
Compute $de = 1 (\mathrm{mod}\ \varphi(nn'))$, get d.
Select random number x_i, $i = 1, \cdots, N-1$.
Compute $x_N = d - \sum_{i=1}^{N-1} x_i$, get $x_i, i = 1, \cdots, N$.
Send $\xi_i = (\mathcal{B}_j, x_i), i = 1, \cdots, N$ to each AC member.
for $i = 1, \cdots, N$ **do**
  Compute h and n' from $hash = (h, n')$.
  Compute $g' \leftarrow H_n(m')$, $g'^{-1} (\mathrm{mod}\ nn')$.
  Compute $r'_i = (hg'^{-1})^{d_i} (\mathrm{mod}\ nn')$.
Broadcast $r'_i, i = 1, \cdots, N-1$.
return $r'_i, i = 1, \cdots, N-1$.

[28] to audit the reasonableness and necessity of the request. After AC reaches a consensus, DID_{U_i} will be revoked by AC performing the DID rewriting operation. Next, AC run Algorithm 3 to rewrite the block $\mathcal{B}_j$.

As described in Algorithm 3, the edit initiator solves $de = 1 (\mathrm{mod}\ \varphi(nn'))$ to get d by et, and then computes the edit information $\xi_i, i = 1, \cdots, N-1$ of each AC member and distributes it to each committee member. Then, each AC member executes independently to obtain a new chameleon random number fragment $r'_i, i = 1, \cdots, N$ and finally broadcast r'_i to all blockchain nodes for verification and revocation.

DID Revocation. After all blockchain nodes receive the $r'_i, i = 1, \cdots, N$, nodes will verify the edit result and make the corresponding edits locally. As shown in Algorithm 4, AC first extracts $hash$ from the blockchain according to the block number $\mathcal{BN}_j$. Then, each blockchain node computes $r' = \prod_i^N r'_i (\mathrm{mod}\ nn')$ and execute $HashCheck(pk, m', r', hash \to \{0\ or\ 1\})$. If

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Revoke the DID on chain

Input: The input chain $\mathcal{C}$, number of DID block $\mathcal{B}_j$, original message m, new message $m^{'}$, chameleon hash value $hash$, chameleon hash ephemeral trapdoor key et, number of nodes N, $H_n[\{0,1\}^* \rightarrow \mathbb{Z}_p]$, blockchain node $\mathcal{BN}_j$, numbers of node M, number of committee nodes N.

Output: The new chain $\mathcal{C}'$.

```
Parse the chain C as (B_1, ··· , B_n).
for Each blockchain nodes BN_j, j = 1, ··· , M do
    Compute r' = ∏_i^N r'_i (mod nn')
    if HashCheck(pk, m', r', hash) then
        Revoke(C) → C'.
return C'.
```

$HashCheck$ outputs 1, each node performs the corresponding editing operation locally, otherwise, the node does not do any processing.

C. Security Analysis

1) Informal Security Analysis: **Single point of failure resistance**: Since FutureDID is designed with the multiple committees' mechanism, there is no central authority, which means that FutureDID can be resilient to attacks or failures from a single entity. Specifically, multiple entities are involved in the issuance of credentials, and these entities are distributed across the FutureDID, which helps to prevent any one entity from having too much power or control. Besides, The DKG [25] algorithm is a cryptographic protocol that allows multiple parties to jointly generate a public key and corresponding private keys, without any one party knowing the private key of another. This can help to ensure that the key used to sign credentials is generated in a secure and decentralized manner. In addition, the Byzantine consensus protocol is a type of algorithm that allows a group of nodes to reach an agreement on a shared state, even if some of the nodes are malicious or faulty. It ensures that FutureDID remains secure and reliable, even if some nodes are compromised. Therefore, FutureDID has the ability to resist a single point of failure.

Mutual verification: In FutureDID, a valid credential $cred$ must contain a signature σ signed by the credential committee. The credential committee is required to execute a Byzantine consensus protocol [28] to verify each committee member's identities, capabilities, and knowledge before issuing credentials. Specifically, assuming the number of credential committee members is n and the Byzantine fault tolerance value is f. If the number of committee members who have not passed mutual verification exceeds f, then the credential committee cannot issue any credential, which is guaranteed by Byzantine consensus [28]. Thus, the execution of mutual verification can be guaranteed through Byzantine consensus, and FutureDID achieves mutual verification.

Claim validity: Assuming that each user sets his claims, the validity of these claims can be directly inferred from the integrity guarantee provided by the W3C recommendations [4], [5], as well as the cryptographic guarantees that underpin FutureDID. These technologies ensure that claims are tamper-evident and cannot be modified without detection, thus enhancing the overall reliability and security of FutureDID.

Credential validity: Assuming that each user holds his credentials, the validity of these credentials can be inferred directly from the integrity guarantee provided by the W3C recommendations [4], [5] and the cryptographic guarantees that underpin FutureDID. These technologies ensure that the credentials are tamper-evident and cannot be modified without detection, thus enhancing the overall security and reliability of FutureDID.

2) Formal Security Analysis: We formalize the expression of security definitions.

Definition 1: Blockchain editability. If FutureDID meets the following formula:

$$\Pr\left[\begin{array}{c} Setup(\lambda) \rightarrow param \\ KeyGen(param) \rightarrow (sk, pk) \\ BlockGen(pk, m) \rightarrow (et, r, hash) \\ Rewrite\left(\mathcal{B}_j, et, m, hash, m'\right) \rightarrow (r_i') \\ HashCheck(pk, m, r, hash) \rightarrow 1 \\ Revoke\left(\mathcal{C}, \mathcal{B}_j, et, m, hash, m'\right) \rightarrow (\mathcal{C}') \end{array}\right] \geq 1 - \varepsilon(\lambda),$$

the blockchain can perform an edit operation, where $\varepsilon(\lambda)$ is an ignorable function.

Definition 2: Public key non-editability. For any polynomial time adversary $\mathcal{A}$, there exists a negligible function $\varepsilon(\lambda)$ satisfying the following equation:

$$\Pr\left[Game_{\mathcal{A}}^{Public}(\lambda) = 1\right] \leq \varepsilon(\lambda),$$

FutureDID is considered to have the public key non-editability.

Definition 3: Private key non-editability. For any polynomial time adversary $\mathcal{A}$, there exists a negligible function $\varepsilon(\lambda)$ satisfying the following equation:

$$\Pr\left[Game_{\mathcal{A}}^{Private}(\lambda) = 1\right] \leq \varepsilon(\lambda),$$

FutureDID is considered to have the private key non-editability.

Definition 4: Unforgeability of identity credentials. For any polynomial time adversary $\mathcal{A}$, there exists a negligible function $\varepsilon(\lambda)$ satisfying the following equation:

$$\Pr\left[Game_{\mathcal{A}}^{forge}(\lambda) = 1\right] \leq \varepsilon(\lambda),$$

FutureDID is considered to have the unforgeability of identity credentials.

Definition 5: Sybil-resistance. For any polynomial time adversary $\mathcal{A}$, if the DID system is meets the following formula:

$$\Pr\left[Game_{\mathcal{A}}^{Sybil}(\lambda) = 1\right] \leq \varepsilon(\lambda),$$

FutureDID can resist sybil attack, where $\varepsilon(\lambda)$ is an ignorable function.

Game 1: The description of $Game^{public}$ is as follows:

1) The challenger $\mathcal{C}$ inputs the security parameter λ, runs the key generation algorithm $KeyGen$ and obtains the public-private key pair (pk^*, sk^*), $Q \leftarrow \phi$ denotes the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

set of queries for the adversary $\mathcal{A}$ to access the hash oracle model.
2) The adversary $\mathcal{A}$ asks hash oracle first, and hash oracle executes: $(hash, r, et) \leftarrow Hash(pk^*, m)$, When $hash = \perp$, return $\perp$, let $Q \leftarrow Q \cup \{(hash, m)\}$, return $(hash, r)$.
3) After completing a series of hash oracle queries, adversary $\mathcal{A}$ selects a generated hash $(hash^*, m^*)$ as the target for which to generate a collision.
4) The adversary $\mathcal{A}$ makes another series of queries to the hash.
5) The adversary $\mathcal{A}$ returns $(m^{*'}, r^{*'})$ as a found collision.
6) When $HashCheck\left(pk^*, m^{*'}, r^{*'}, hash^*\right) = 1 \wedge m^{*'} \notin Q \wedge m^{*'} \neq m^*$, it returns 1, otherwise it returns 0.

Game 2: The description of $Game^{Private}$ is as follows:
1) The challenger $\mathcal{C}$ inputs the security parameter λ, runs the key generation algorithm $KeyGen$ and obtains the public-private key pair (pk^*, sk^*), $Q \leftarrow \phi$ denotes the set of queries for the adversary $\mathcal{A}$ to access the edit oracle model.
2) The challenger $\mathcal{C}$ pre-generates a series of messages $(m_1, ..., m_n)$ and computes the corresponding chameleon hash, denoted $U = (hash_1, ..., hash_n)$. When the adversary $\mathcal{A}$ queries the edit oracle, the oracle input are sk^*, m, m', r, et, $hash$, when $HashCheck\,(pk^*, m, r, hash) = 0$, the edit oracle returns $\perp$, otherwise the algorithm is executed sequentially $r'_i \leftarrow Rewrite\,(m, m', r, hash)$, return r'.
3) After $\mathcal{A}$ completes edit oracle queries, it returns $(m^*, r^*, m^{*\prime}, r^{*\prime})$ as the found collision.
4) When $HashCheck\left(pk^*, m^{*'}, r^{*'}, hash^*\right) = 1 \wedge m^{*'} \notin Q \wedge m^{*'} \neq m^*$, it returns 1, otherwise it returns 0.

Game 3: The description of $Game^{forge}$ is as follows:
1) The challenger $\mathcal{C}$ inputs the security parameter λ, runs the key generation algorithm $KeyGen$ and obtains the key pair (pk^h, sk^h), send the public key pk^h to the adversary $\mathcal{A}$.
2) The adversary $\mathcal{A}$ runs the encryption algorithm several times with the obtained public key pk^h to obtain a series of credentials $(c_1, ..., c_n)$.
3) The challenger $\mathcal{C}$ generates a random credential v and sends it to the adversary $\mathcal{A}$. The adversary $\mathcal{A}$ chooses the credential in his hand to pair with the challenge credential v and generates the corresponding proofs π for the pairs.
4) The challenger $\mathcal{C}$ first verifies the correctness of the proof π. When the verification fails, challenger $\mathcal{C}$ returns 0, and when the verification passes, the challenger $\mathcal{C}$ uses the private key sk_h to decrypt the credential signature, and determine whether the decrypted pairs of credentials are equal, if not equal to return 1, equal to return 0.

Game 4: The description of $Game_{\mathcal{A}}^{Sybil}(\lambda)$ is as follows:
1) The adversary $\mathcal{A}$ inputs the security parameter λ, runs the key generation algorithm $KeyGen$ and obtains the key pair (pk^C, sk^C).
2) The adversary $\mathcal{A}$ initials a register information R such as (id_0, a, v_0). These R represent the identity information, where id_0 is the identifier, a is the attribute (such as the Social Security number), and v is the value of the attribute.
3) The adversary $\mathcal{A}$ sends R to the register oracle for registering his DID.
4) The register oracle checks whether $v_0 \in \mathsf{IDTable}$.
5) If $v_0 \in \mathsf{IDTable}$, the register oracle returns 1, otherwise, it returns 0.

Next, we give the formal security proof of FutureDID.

Theorem 1: Blockchain editability. When the administration committee (AC) approves the identity revocation request through the DID, find the temporary trapdoor $et = (p', q')$ corresponding to the block combined with its chameleon hash private key sk to generate the block editing auxiliary information d, through the committee as a whole to generate the chameleon random number r', to ensure that before and after the modification of the consistency of the block's hash.

Proof: After AC approves the identity revocation request, AC computes

$$\varphi(nn') = (p-1)(q-1)(p'-1)(q'-1).$$

AC calculates the congruence $de = 1(\mathrm{mod}\ \varphi(nn'))$, and gets d. AC executes the $share$ algorithm to fragment the auxiliary editing information d, and get $d = d_1 + d_2 + \cdots + d_N$. Each AC member receives the auxiliary editing information fragments, calculates $g' \leftarrow H_{nn'}(m')$, then computes $g'^{-1}(\mathrm{mod}\ nn')$, calculates $r'_j = \left(hg'^{-1}\right)^d (\mathrm{mod}\ nn')$, and finally aggregates different r'_j by each node, and there the following equation holds,

$$r' = \prod_{j=1}^{N} r'_j\,(\mathrm{mod}\ nn') = \left(hg'^{-1}\right)^d (\mathrm{mod}\ nn').$$

Each node calculates based on chameleon hash algorithm, $hash = (h, n') = (gr^e(\mathrm{mod}\ nn'), n')$, thus the following equation holds,

$$h' = g'r'^e(\mathrm{mod}\ nn') = g'hg'^{-1}(\mathrm{mod}\ nn') = h.$$

Therefore, when the AC uses the new random number r', the hash value of the block before and after can be kept unchanged, so the blockchain can be editable. □

Theorem 2: Public key non-editability. If the integer factorization problem is hard, and any probabilistic polynomial-time adversary (PPT) $\mathcal{A}$ cannot solve the integer factorization problem, then under the random oracle model, FutureDID has public key non-editability.

Proof: Assuming that FutureDID does not have public key non-editability, so there exists a probabilistic polynomial-time adversary $\mathcal{A}$ who has a non-negligible probability of winning the $Game^{Public}$, then one can construct an adversary $\mathcal{B}$ to solve the integer factorization problem with non-negligible probability.

Adversary $\mathcal{B}$ simulates a hash oracle to interact with adversary $\mathcal{B}$. Firstly, the cryptographic exponent e is obtained from

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

a challenger as a parameter of the hash oracle, and the RSA modulus n is used as the public key of the hash oracle.

Regarding the first stage of the adversary $\mathcal{A}$'s query, for the computation of $H_{nn'}(m)$, the adversary $\mathcal{B}$ obtains a challenge $c \in Z_n^*$ from the RSA challenger and asks $c \in Z_{nn'}^*$ to take c as the value of $H_{nn'}(m)$, chooses the random number r, and the random oracle returns it:

$$((H_{nn'}(m)r^e(\text{mod } nn'), n'), r) = ((cr^e(\text{mod } nn'), n'), r).$$

Suppose that adversary $\mathcal{A}$ identifies the collision target as $((h^*, n'), r^*) = ((H_{nn'}(m^*)r^{*e}(\text{mod } nn'), n'), r^*)$, in the subsequent query, for the computation of $H_{nn'}(m_i)$, adversary $\mathcal{B}$ random chooses $v_i \in Z_{nn'}^*$, takes v_i^e as the value of $H_{nn'}(m_i)$, chooses the random number r_i, and the random oracle returns:

$$((H_{nn'}(m_i)r_i^e(\text{mod } nn'), n'), r_i) = ((v_i^e r_i^e(\text{mod } nn'), n'), r_i).$$

By assumption, adversary $\mathcal{A}$ has non-negligible probability output $(m^{*'}, r^{*'})$ and satisfies:

$$HashCheck(pk^*, m^{*'}, r^{*'}, hash^*) = 1 \wedge m^{*'} \notin Q \wedge m^{*'} \neq m^*.$$

Since adversary $\mathcal{A}$ completes all the random oracle accesses before outputting the result, we have

$$H_{nn'}(m^*)r^{*e} = H_{nn'}(m^{*'})r^{*'e}(\text{mod } nn').$$

Thus, we get $cr^{*e} = v^{*'e}r^{*'e}(\text{mod } nn')$, which solves for $c = (v^{*'}r^{*'}r^{*-1})^e(\text{mod } nn')$, and so

$$c = (v^{*'}r^{*'}r^{*-1})^e + k(nn'), k \in Z.$$

Modulo public key n on both sides of the equation yields $c = (v^{*'}r^{*'}r^{*-1})^e(\text{mod } n)$, so that the solution to c is $v^{*'}r^{*'}r^{*-1}(\text{mod } n)$.

With all but negligible probability, a PPT adversary $\mathcal{A}$ can produce $v^{*'}r^{*'}r^{*-1}(\text{mod } n)$, because decomposing the factors of a large number is computationally impossible [30]. Therefore our system is secure and has the public key non-editability. □

Theorem 3: Private key non-editability. If the integer factorization problem is hard, any probabilistic polynomial-time adversary $\mathcal{A}$ cannot solve the integer factorization problem, then under the random oracle model, FutureDID has private key non-editability.

Proof: Assuming that FutureDID does not have private key non-editability, so there exists a probabilistic polynomial-time adversary $\mathcal{A}$ who has a non-negligible probability of winning the $Game^{Private}$, then one can construct an adversary $\mathcal{B}$ to solve the integer factorization problem with non-negligible probability.

Adversary $\mathcal{B}$ simulates a hash oracle to interact with adversary $\mathcal{A}$. Firstly, the cryptographic exponent e is obtained from a challenger as a parameter of the edit oracle, and the RSA modulus n' as a parameter of the temporary trapdoor of the edit oracle model.

First, adversary $\mathcal{B}$ generates a series of messages and computes the corresponding chameleon hashes, where for the $H_{nn'}(m_i)$ computation, adversary $\mathcal{B}$ obtains a challenge ciphertext $c_i \in Z_{n'}^*$ from the challenger, and asks for $c_i \in Z_{nn'}^*$, which will be used as the value of $H_{nn'}(m)$ only.

When adversary $\mathcal{A}$ accesses the random oracle, $\mathcal{B}$ randomly selects $v \in Z_{nn'}^*$, using v^e as the value of $H_{nn'}(m)$. Adversary $\mathcal{A}$ chooses random number r_i, and gets the corresponding chameleon hash value:

$$((H_{nn'}(m_i)r_i^e(\text{mod } nn'), n'), r_i) = ((c_i r_i^e(\text{mod } nn'), n'), r_i).$$

Adversary $\mathcal{A}$ requests to change some information in the query phase, for example, request to change m_j to m_j', set $H_{nn'}(m_j') = v_j'^e$, then need to determine the return value r_j', according to the relationship between the two chameleon hash value is equal, there is $c_j r_j^e = v_j'^e r_j'^e$, the adversary $\mathcal{B}$ for c_j query the oracle to get the corresponding m_j, calculate the $r_j' = v_j'^{-1} m_j r_j$ and return the result to the adversary $\mathcal{A}$.

At the end of adversary $\mathcal{A}$'s visit, the adversary has a non-negligible probability of outputting $(m^*, r^*, m^{*'}, r^{*'})$ and satisfying.

$$HashCheck(pk^*, m^*, r^*, hash^*) = 1 \wedge HashCheck(pk^*, m^{*'}, r^{*'}, hash^*) = 1 \wedge m^* \notin Q \wedge m^* \neq m^{*'}.$$

So there we have it,

$$H_{nn'}(m^*)r^{*e} = H_{nn'}(m^{*\prime})r^{*\prime e}(\text{mod } nn').$$

Thus we have $cr^{*e} = v^{*\prime e}r^{*\prime e}(\text{mod } nn')$, which solves for $c = (v^{*\prime}r^{*\prime}r^{*-1})^e(\text{mod } nn')$, so $c = (v^{*\prime}r^{*\prime}r^{*-1})^e + k(nn'), k \in Z$, and both sides of the equation modulo n' yield $c = (v^{*\prime}r^{*\prime}r^{*-1})^e(\text{mod } n')$, thus yielding a solution of c for $v^{*\prime}r^{*\prime}r^{*-1}(\text{mod } n')$.

With all but negligible probability, a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can produce $v^{*'}r^{*'}r^{*-1}(\text{mod } n')$, because decomposing the factors of a large number is computationally impossible [30]. Therefore our system is secure and has the private key non-editability. □

Theorem 4: Unforgeability of identity credentials. If the computational ensemble residual problem is hard, then a malicious user cannot forge identity credentials to allow false identity credentials to be verified, i.e., the scheme has identity credential unforgeability.

Proof: Assuming that there exists a malicious user, i.e., adversary $\mathcal{A}$, who can win the game $Game^{forge(\lambda)}$ with non-negligible probability, i.e., $\Pr[Game^{forge(\lambda)} = 1] \geq \varepsilon(\lambda)$, we can leverage on the ability of adversary $\mathcal{A}$ to construct probabilistic algorithms for solving the computational ensemble residual problem.

First compute the public-private key pairs obtained by the $KeyGen$ algorithm: $pk_i = (n_h, g), sk_h = \gamma$, send the public key (n_h, g) to adversary $\mathcal{A}$. Adversary $\mathcal{A}$ uses the obtained public key (n_h, g) to execute an encryption algorithm on a freely chosen series of values $(m_1, \cdots, m_t)$, let the obtained series of ciphertexts be:

$$(c_i, \cdots, c_t), c_i = g^{m_i} r_i^{n_h}(\text{mod } n_h^2).$$

Generate a combinatorial residual problem: $y_h^n = z(\text{mod } n_h^2)$, where z is known and the value of y needs to be determined. Generate a challenge v based on the list of ciphertexts

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

of the adversary $\mathcal{A}$, which satisfies $c_1 v = z (\mathrm{mod}\ n_h^2)$, i.e., $v = z c^{-1} (\mathrm{mod}\ n_h^2)$, and send it to the adversary $\mathcal{A}$, assume $v = g^{m_0} r_0^{n_h} (\mathrm{mod}\ n_h^2)$.

Let the output of the adversary be $(c_j v, pf)$, and by assumption, adversary $\mathcal{A}$ can be falsified with non-negligible probability, i.e., $\Pr[m_j + m_0 \neq 0, pf^{n_h} = c_j^v] \geq \varepsilon(\lambda)$. when $j = 1$, we have the following equation: $c_j^v = c_1^v = z (\mathrm{mod}\ n_h^2)$, and thus pf is the solution to the conjugate residual problem.

Let the probability that we solve the chi-squared residual problem be $\Pr[solve]$, so we have:

$$\Pr[solve] = \Pr[m_j + m_0 \neq 0, pf^{n_h} = c_j^v, j = 1] \geq \frac{1}{t}\varepsilon(\lambda).$$

Therefore the probability that we solve the combinatorial residual problem is not negligible. It is thus proved that our system has the unforgeability of identity credentials. □

Theorem 5: Sybil resistance. When an adversary $\mathcal{A}$ applies to re-register his identity in FutureDID and submits his register information $R = \{id, a, v\}$, where id is the identifier, a is the attribute related the real-world identity, and v is the value of the attribute. $\mathcal{A}$ is allowed to register repeatedly with a negligible probability, then FutureDID has Sybil resistance.

Proof: FutureDID first retrieves the latest IDTable from the blockchain. Next, FutureDID checks whether the $\mathcal{A}$'s attribute value v exists in the IDTable. Since v is issued by authorities of the real world (e.g., governments, international organizations), $\mathcal{A}$ can obtain multiple different v with a negligible probability. Therefore, there is a negligible probability that v cannot be found on IDTable. Then, FutureDID returns 1, indicating that the attribute value v is already associated with a DID. Finally, FutureDID does not allow $\mathcal{A}$ to re-register a different DID. Therefore, FutureDID has the capability of Sybil resistance. □

VI. Implementation and Evaluation

In this section, we introduce the design of experiments and evaluate the performance of FutureDID.

A. Implementation

- **Environment**: We implement the FutureDID identity management prototype system based on the FISCO BCOS [31] blockchain. Specifically, FutureDID is implemented in Java with more than 1800 lines of code. We take SHA256 as the default hash function and 2048-bit RSA group for the chameleon hash. Next, FutureDID is deployed on a 2.3GHz Intel Core i5 4-core computer with 16G memory. By default, four fully functional nodes are deployed. The operating system is Ubuntu 16.04.7 LTS 64-bit, the blockchain platform is FISCO BCOS 2.0 [31]. The network response time between nodes is less than 20 ms.
- **System APIs**: FutureDID API includes nine interfaces, six of which are core functions: creatID(), claim(), distributedKey(), issueCred(), verify(), idenitityRevok(). The functions of the APIs are shown in Fig 4.

FutureDID

- setup(): Initial the whole system.
- createId(): Register identity on FutureDID, and return the DID.
- keyGen(): Key generation. Could be instantiated by the user locally or by the committee nodes in a distributed fashion.
- claim(): Generate a claim about the identity.
- distributedKey(): Distributed key generation.
- issueCred(): Issue credentials.
- verify(): Verify that the received credentials.
- idenitityRevok(): Revoke the DID in FutureDID.

Fig. 4. System APIs.

- **Experiment scenarios**: To showcase the capabilities of FutureDID, we conducted stress testing experiments to verify FutureDID's key APIs. We then simulated the process of issuing credentials to a user after the credential committee authenticates the issuer. Additionally, we simulated the process of revoking the identity of a sanctioned user by the administration committee, which requires removing their identity.

B. System Performance Detail

As shown in Fig. 4, the key functional APIs of FutureDID are createId(), claim(), distributedKey(), issueCred(), verify(), idenitityRevok(). In order to be able to more realistic simulation system capabilities, we perform the stress test of FutureDID APIs with 500 concurrent data used for data acquisition, and the stress test lasted for 5 minutes. The Jmeter [32] stress test software sends the transaction to each blockchain node. The results are shown in Table II.

1) createId()**:** This API function is to create public and private keys, register DIDs on the chain, and return the public key, private key, and DID. The result shows that createId() can complete 176 tasks per second with high availability.
2) claim()**:** This API function is to convert the user's claim data into FutureDID's format specification, also known as "standardized formatted data". The issuance of any credential requires the conversion of the issuer will then issue the credential based on the "standardized formatted data". The result shows that claim() can complete 2,158 tasks per second and the minimum response is only 6 ms.
3) distributedKey()**:** This API is an implementation of distributed key generation, storing the public key on the blockchain. The main steps of distributed key generation are the initialization of the credential committee(initialization), key generation of the committee members (generation), and extraction of the public key (extraction). As the results show in Table II, the average response time is 2412 ms, and transactions per second are 48.
4) issueCred()**:** This API issues credentials based on the previously distributed key generation. If the user claim

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STRESS TEST OF SYSTEM KEY APIS

APIs	Number of Transactions	Average Response (ms)	Minimum Response (ms)	Maximum Response (ms)	Transactions per Second
createId()	52,812	2,212	201	7,974	176
claim()	64,755	185	6	1,276	2,158
distributedKey()	14,487	2,412	1,433	5,842	48
issueCred()	934,236	103	3	10,184	3,114
verify()	469,847	280	21	1,522	1,566
identityRevoke()	152,165	756	412	1643	507

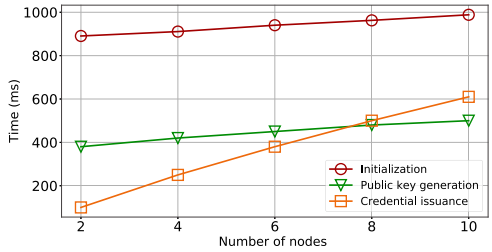


Fig. 5. Running time for credential issuance.

TABLE III
RUNNING TIME OF IDENTITY REVOCATION

Num. of Nodes	Setup	Block Generation	Rewiriting DID	Revoke DID
2	521.6 ms	14.3 ms	21.2 ms	53.4 ms
4	582.3 ms	16.5 ms	32.8 ms	61.5 ms
8	644.1 ms	24.9 ms	38.9 ms	87.6 ms
16	911.2 ms	45.1 ms	43.8 ms	112.1 ms
32	1,307.5 ms	82.1 ms	53.3 ms	164.4 ms
64	1,859.5 ms	13.48 ms	82.4 ms	219.3 ms

TABLE IV
SYSTEM ROBUSTNESS EVALUATION

Number of Compromised Members	Identity Subsystem	Revocation Subsystem
1	✔	✔
2	✔	✔
4	✔	✔
8	✘	✘
16	✘	✘

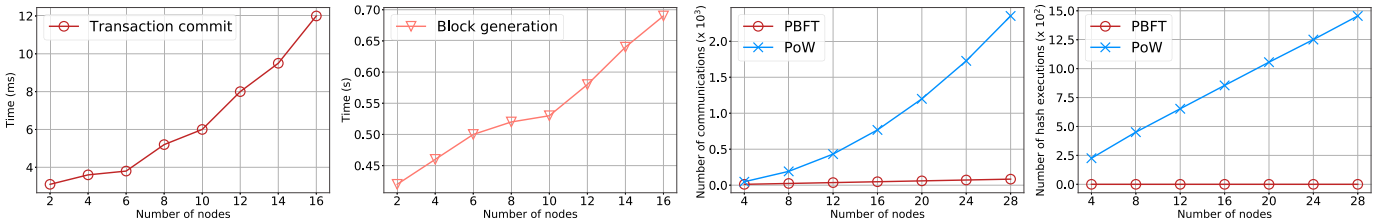
passes the authentication of the committee, the committee will issue the credentials. As shown in Table II, FutureDID can issue an average of 3,114 credentials per second. Additionally, to verify the effect of different sizes of committees on the running time of the credential issuing algorithm. We have done several experiments based on different committee sizes n, and the specific number of committees is $n = 2$, $n = 4$, $n = 6$, $n = 8$, and $n = 10$. The results are shown in Fig 5. In the initialization phase (red line), the running time increases with the number of nodes, and the computational overhead is 940.3 ms when $n = 8$. In the generation phase of the public key (blue line), the running time also increases with the number of nodes, when the $n = 8$, the computational cost is 410.9 ms. In the issuance phase (yellow line), the larger the number of nodes the slower the extraction of the public key, when the number of $n = 8$, the computational cost is 380.6 ms.

5) verify(): The process of verifying a credential is that the verifier verifies the credential by the public key of the credential committee. If the verification passes, the verifier trusts the credential, and vice versa. Therefore, the average time to verify a credential is shown in Table II and is 280 ms, and it can verify 1,566 credentials per second.
6) idenitityRevok(): There are four main steps in identity revocation, which are **System Setup**, **Block Generation**, **DID Rewriting**, **DID Revocation**. The results showed that FutureDID was able to handle 507 revocation operations per second, with an average response time of milliseconds. In addition, to verify the computational overhead on each step of our identity revocation API, we conducted experiments with different numbers of node. The security parameter is 256, that is, the chameleon hash value, and the chameleon hash public and private key is 256 bits. The results are shown in Table III. It can be seen from the results that the overall execution rate of the identity revocation algorithm is millisecond level. The most time-consuming is the setup function, when the $n = 16$ is 911.2 ms, and other functions are basically around 100 ms. Thus, The identity revocation API has less impact on the blockchain because the setup function only needs to be performed once when the blockchain is initialized.

Robustness/ Single point of failure resistance of FutureDID. We also simulated the scenario where committee nodes in our system are compromised by adversary attacks. When the MC and AC committees consist of 20 members each, denoted as $n = 20$, the experimental results are presented in Table IV, where "✔" indicates FutureDID's normal operation and "✘" represents the inability to function properly. From the experimental results, it can be observed that when the number of compromised committee members is $t > n/3$, FutureDID cannot function properly. However, when it is $t < n/3$, FutureDID operates normally, the experiments demonstrate that FutureDID has a single point of failure resistance and robustness.

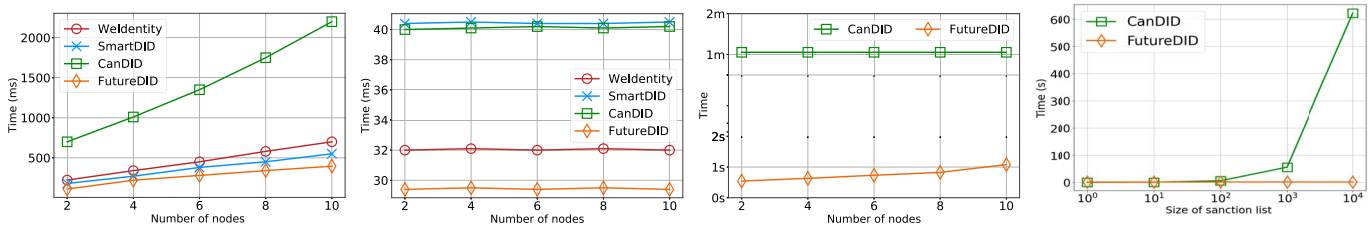
Perfomance of Blockchain. We evaluated the performance data of the FISCO BCOS blockchain that we used. We examined the performance of the underlying blockchain based on transaction commit overhead, block generation overhead, the complexity of consensus protocol communication, and the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.



(a) Running time for transaction commit.
(b) Running time for block generation.
(c) Communication complexity comparison.
(d) Computational complexity comparison.

Fig. 6. Perfomance of blockchain.



(a) Running time for credential generation.
(b) Running time for verification.
(c) Running time for revocation with different nodes.
(d) Running time for revocation with different data size.

Fig. 7. Perfomance of FutureDID.

number of consensus protocol hash executions. Fig. 6(a) and 6(b) illustrate the block generation time and transaction commit time of FutureDID in a blockchain network with varying numbers of consensus nodes. As the number of consensus nodes increases, the block generation time exhibits minor oscillations and a slight upward trend. This can be attributed to the communication overhead between the nodes and the PBFT consensus algorithm. Fig. 6(c) and 6(d) illustrate the PBFT consensus protocol used by FutureDID and compare the traditional PoW consensus protocol in terms of communication complexity and hash executions. The communication complexity of PoW is $O(N)$, whereas the communication overhead of PBFT is $O(N^2)$. When the number of nodes is small, the communication overhead of PoW is greater than that of PBFT. However, as the number of nodes continues to increase, the communication complexity of PBFT will far exceed that of PoW. The hashing overhead of PoW is proportional to the number of nodes, and the hashing overhead of PBFT is much smaller than that of PoW. The hashing overhead of PBFT is also proportional to the number of nodes, but the arithmetic consumption of PBFT is very small compared to PoW, which is almost negligible.

C. Computational Cost Comparison

We compare the performance of FutureDID with WeIdentity [7], SmartDID [8], and CanDID [9] since they represent the state-of-art DID system from industry to academia. We run all systems in the same environment which is the same as Section VI-A. The experimental results are shown in Fig. 7.

Computational costs in the identity subsystem. The experimental result is illustrated in Fig. 7, we perform the comparison experiments in *credential generation* and *verification* phases, the results in Fig. 7(a) and 7(b) shown that computational costs of FutureDID are 394.46 ms and 29.24 ms, respectively, which has the best performance. As CanDID [9] and SmartDID [8] adopt ZKP technology in the credential generation phase, therefore, they need extra time in these two phases.

Computational costs in the revocation subsystem. Furthermore, we also experimentally evaluated the revocation subsystem, but WeIdentity [7] and SmartDID [8] lack revocation functionality, so our comparison object was only CanDID [9]. As shown in Fig. 7(d) and 7(e), we conducted two separate experiments with varying numbers of nodes and sanction list sizes, respectively. We use the OFAC sanctions list [33] as our dataset. When the size of the sanction list is 1000 strings, as Fig. 7(d) shows that the computational overhead of CanDID [9] is stable at about 1 minute and does not change with the increase of the number of nodes, while the computational overhead of FutureDID increases with the increase of the number of nodes, and is 1.08 s when the number of nodes is 10, which is more than 60 times less than CanDID [9]. Besides, the experimental results are described in Fig. 7(e), as the size of the sanctions list increases, the computational cost of CanDID [9] increases exponentially. This is because the revocation mechanism of CanDID [9] requires proving the user's non-existence in the sanctions list, and the larger the sanctions list, the higher the calculation cost. In contrast, FutureDID directly deletes users in the sanctions list from the blockchain, which is a one-time operation and does not depend on the dataset. Therefore, the time cost of FutureDID remains stable at around 1.31 s.

In summary, the experimental results show that FutureDID achieves the best efficiency improvement on average compared with state-of-the-art DID systems. In particular, FutureDID's time overhead is at least 60 $\times$ higher than CanDID's [9] revocation capability.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.

VII. Conclusion

FutureDID is a decentralized identity management system. We have identified limitations with the W3C DID recommendation for lack of protection against single points of failure and missing practical revocation mechanisms. To address these issues, we have developed a decentralized identity system that includes an identity subsystem and a revocation subsystem. To achieve decentralized credential issuance through multi-party validation, we utilize distributed key generation and Byzantine consensus protocols. Additionally, we have developed an editable blockchain system based on chameleon hashing technology to enable identity revocation. Extensive experiments have been conducted to verify the efficiency and functionality of our system. The results demonstrate that our system provides superior functionality and security compared to the latest existing schemes.

References

[1] M. Kuperberg, "Blockchain-based identity management: A survey from the enterprise and ecosystem perspective," *IEEE Trans. Eng. Manage.*, vol. 67, no. 4, pp. 1008–1027, Nov. 2020.

[2] "OpenID foundation website." OpenID. Accessed: May 15, 2024. [Online]. Available: https://openid.net/

[3] A. Parsovs, "Estonian electronic identity card: Security flaws in key management," in *Proc. USENIX Secur.*, California, USA: USENIX Association, 2020, pp. 1785–1802.

[4] "Decentralized identifiers (dids) v1.0." W3C. Accessed: May 15, 2024. [Online]. Available: https://www.w3.org/TR/did-core/

[5] "Verifiable credentials data model v2.0." W3C. Accessed: May 15, 2024. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/

[6] "Tencent cloud decentralized identity." Tencent. Accessed: May 15, 2024. [Online]. Available: https://cloud.tencent.com/product/tdid

[7] "Weidentity documentation." Webank. Accessed: May 15, 2024. [Online]. Available: https://weidentity.readthedocs.io/en/latest/

[8] J. Yin et al., "SmartDID: A novel privacy-preserving identity based on blockchain for IoT," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6718–6732, Apr. 2023.

[9] D. Maram et al., "CanDID: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1348–1366.

[10] R. Chotkan, J. Decouchant, and J. Pouwelse, "Distributed attestation revocation in self-sovereign identity," in *Proc. IEEE Local Comput. Netw.,* 2022, pp. 414–421.

[11] D. P. Ahn and R. D. Ludema, "The sword and the shield: The economics of targeted sanctions," *Eur. Econ. Rev.*, vol. 130, 2020, Art. no. 103587.

[12] L. Tauginienė and V. Jurkevicius, "Incongruences of ethical and legal norms in academia: The case on revocation of doctoral degrees," *J. Academic Ethics*, vol. 15, pp. 73–91, Mar. 2017.

[13] S. Khan et al., "A privacy-preserving and transparent identity management scheme for vehicular social networking," *IEEE Trans. Veh. Technol.*, vol. 71, no. 11, pp. 11555–11570, Nov. 2022.

[14] M. R. Ahmed, A. K. M. M. Islam, S. Shatabda, and S. Islam, "Blockchain-based identity management system and self-sovereign identity ecosystem: A comprehensive survey," *IEEE Access*, vol. 10, pp. 113436–113481, 2022.

[15] Y. Yu, Z. Li, Y. Tu, Y. Yuan, Y. Li, and Z. Pang, "Blockchain-based distributed identity cryptography key management," in *Proc. Int. Conf. Comput. Res. Develop. (ICCRD)*, 2023, pp. 236–240.

[16] S. Nakamoto and A. Bitcoin, "A peer-to-peer electronic cash system," *Bitcoin,* vol. 4, no. 2, 2008, Art. no. 15.

[17] L. Zhu, Q. Liu, Z. Chen, C. Zhang, F. Gao, and Z. Yang, "A novel covert timing channel based on bitcoin messages," *IEEE Trans. Comput.*, vol. 72, no. 10, pp. 2913–2924, Oct. 2023.

[18] N. Naik and P. Jenkins, "uPort open-source identity management system: An assessment of self-sovereign identity and user-centric data platform built on blockchain," in *Proc. IEEE Int. Symp. Syst. Eng. (ISSE),* 2020, pp. 1–7.

[19] Y. Liu et al., "A blockchain-based decentralized, fair and authenticated information sharing scheme in zero trust internet-of-things," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 501–512, Feb. 2023.

[20] J. Liu et al., "Leakage-free dissemination of authenticated tree-structured data with multi-party control," *IEEE Trans. Comput.*, vol. 70, no. 7, pp. 1120–1131, Jul. 2021.

[21] S. Huh, M. Shim, J. Lee, S. Woo, H. Kim, and H. Lee, "Did we miss anything?: Towards privacy-preserving decentralized ID architecture," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4881–4898, Nov./Dec. 2023.

[22] A. Lei et al., "A blockchain based certificate revocation scheme for vehicular communication systems," *Future Gener. Comput. Syst.*, vol. 110, pp. 892–903, Sep. 2020.

[23] Y. Guo, Z. Lu, H. Ge, and J. Li, "Revocable blockchain-aided attribute-based encryption with escrow-free in cloud storage," *IEEE Trans. Comput.*, vol. 72, no. 7, pp. 1901–1912, Jul. 2023.

[24] C. Zhao et al., "Secure multi-party computation: Theory, practice and applications," *Inf. Sci.*, vol. 476, pp. 357–372, Feb. 2019.

[25] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," *J. Cryptol.*, vol. 20, no. 1, pp. 51–83, 2007.

[26] H. Krawczyk and T. Rabin, "Chameleon signatures," in *Proc. Netw. Distrib. Syst. Secur. (NDSS),* Internet Soc., 2000.

[27] J. Camenisch, D. Derler, S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-Hashes with ephemeral trapdoors - and applications to invisible sanitizable signatures," in *Proc. Public-Key Cryptogr.,* vol. 10175, Springer-Verlag, 2017, pp. 152–182.

[28] J.-P. Martin and L. Alvisi, "Fast byzantine consensus," *IEEE Trans. Dependable Secure Comput.*, vol. 3, no. 3, pp. 202–215, Jul. /Sep. 2006.

[29] F. Calogero, "Lagrangian interpolation and differentiation," *Lettere al Nuovo Cimento (1971–1985)*, vol. 35, no. 9, pp. 273–278, 1982.

[30] D. R. L. Brown, "Breaking RSA may be as difficult as factoring," *J. Cryptol.*, vol. 29, no. 1, pp. 220–241, 2016.

[31] "Financial services blockchain consortium." fisco. Accessed: May 15, 2024. [Online]. Available: https://www.fisco.com.cn/en/

[32] "Apache JMeter." Apache. [Online]. Available: https://jmeter.apache.org/

[33] "Sanctions list search." OFAC. Accessed: May 15, 2024. [Online]. Available: https://sanctionssearch.ofac.treas.gov/



Haotian Deng (Student Member, IEEE) received the M.S. degree from Clemson University, in 2021. He is currently working toward the Ph.D. degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include blockchain, IoT security, and applied cryptography.



Jinwen Liang (Member, IEEE) received the B.S. and Ph.D. degrees from Hunan University, China, in 2015 and 2021, respectively. He is a Postdoctoral Fellow with the Department of Computing of The Hong Kong Polytechnic University. His research interests include applied cryptography, AI security, blockchain, and database security. He served as the Technical Program Committee Chair of the IEEE FMobile 2022, Publicity Chair of the IWCSS 2022, and TPC Member of IEEE VTC' 19 Fall.



Chuan Zhang (Member, IEEE) received the Ph.D. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2021. He is currently an Assistant Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include secure data services in cloud computing, applied cryptography, machine learning, and blockchain.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.



Ximeng Liu (Senior Member, IEEE) received the B.Sc. degree in electronic engineering and the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2010 and 2015, respectively. He is currently a Professor with the College of Mathematics and Computer Science, Fuzhou University, China. He is also a Research Fellow with the School of Information System, Singapore Management University, Singapore. His research interests include cloud security, applied cryptography, and big data security.



Liehuang Zhu (Senior Member, IEEE) received the Ph.D. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2004. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include security protocol analysis and design, group key exchange protocols, wireless sensor networks, and cloud computing.



Song Guo (Fellow, IEEE) is a Full Professor in the Department of Computer Science and Engineering at Hong Kong University of Science and Technology. He also holds Changjiang Chair Professor in Wuhan University. He made fundamental and pioneering contributions to the development of edge AI and cloud-edge computing which has created significant impact from generation of new scientific knowledge to creation of innovative technologies. He published many papers in top venues and received over a dozen Best Paper Awards from IEEE/ACM conferences, journals and technical committees. He is also the recipient of Gold Medal in 2023 Geneva Inventions Expo and Intellectual Property Ambassador Award in 2020 Hong Kong Social Enterprise Competition. He is a Fellow of the Canadian Academy of Engineering, Member of Academia Europaea, and Fellow of the IEEE. He has served on IEEE Fellow Evaluation Committee for both ComSoc and Computer Society. He is the founding and current Editor-in-Chief of IEEE Open Journal of the Computer Society and a member of Steering Committee of IEEE TCC. He has been named on editorial board of a number of prestigious international journals like IEEE TC, IEEE TPDS, IEEE TCC, etc. He has also served as chair of organizing and technical committees of numerous IEEE/ACM conferences and workshops. He has served on RGC engineering panel and been frequently invited for various national and international grant/award reviews.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 10:14:20 UTC from IEEE Xplore. Restrictions apply.